

遙測與空間資訊之分析與應用 - Week 3

國家災害防救科技中心 資訊組

台灣大學土木工程學系 教授

蘇文瑞

wraysu@ntu.edu.tw

The “Glass Box” Philosophy

Student Feedback: “The AI writes code, but I don’t know what’s happening.”

- **Jupyter Notebooks (.ipynb) The Solution:**
Write a cell → Execute → Inspect `gdf.head()` → Visualize `gdf.explore()`
- The AI is your navigator; you are the captain.
Watch the cell numbers [1][2][3]... — they represent your logic sequence.

Goal: Understand the state of your data at every step.

→ *Notebook Cell [1]: Environment Initialization*

GeoPandas: The “Swiss Army Knife” of GIS

- Heavy software (QGIS / ArcGIS) Traditional GIS:
- Lightweight, programmable, and reproducible Vibe Coding:

Core Idea:

It's a Pandas DataFrame with a “Geometry” column that understands math AND geography.

- What can it do?

Read, transform, buffer, overlay, join, export — all in Python

DataFrame + Geometry + CRS = GeoDataFrame

Anatomy of a GeoDataFrame (GDF)

- Columns like name, population, id Attributes:
- The column that stores Points, Lines, or Polygons Geometry:
- The “Map Reference” (Where is this on Earth?) CRS:

Captain’ s Intuition:

If you lose the CRS, your map floats in space.

→ *Notebook Cell [2]: Load Taiwan townships, inspect geometry and CRS*

display(townships.head()) → See MultiPolygon + GCS_TWD97 (degrees)

The Magic of Step-by-Step Analysis

In Jupyter, we break logic into 5 steps:

- Read files (OSM, GeoJSON, CSV) 1. Ingest:
- Change CRS (degrees → meters) 2. Transform:
- Buffer, Intersect, or Dissolve 3. Operate:
- Spatial Join with other layers 4. Analyze:
- Save to CSV / GeoJSON 5. Export:

Each step lives in its own cell. Each step can be inspected.

GeoPandas Power 1: Reprojection

- WGS84 (EPSG:4326) uses degrees. You cannot buffer "500m" in degrees. Problem:
- `gdf.to_crs(epsg=3826)` → TWD97 (Taiwan) Logic:

The Shift:

Geometry column changes from (121.5, 25.0) to (300000, 2700000)

The former is "degrees" ; the latter is "meters"

→ *Notebook Cell [3]: Observe the coordinate values before and after conversion*

Question: Why did the numbers jump from ~121 to ~300,000?

GeoPandas Power 2: Geometric Properties

- `gdf.area` (Calculate the size of a flood zone) Area:
- `gdf.length` (Calculate the length of a fault line) Length:
- `gdf.centroid` (Find the heart of a city polygon) Centroid:

Vibe Check: Use these to audit AI results.

Does the area look reasonable? Is the centroid inside the polygon?

→ *Notebook Cell [4]: Calculate country areas, find the Top 5 largest*

Diagnostic: What happens if you calculate area in EPSG:4326?

GeoPandas Power 3: The Buffer Zone

- Creates a polygon around any geometry at distance d What it does:
- “How many houses are within 500m of this high-risk river?” Disaster App:

Code: `gdf.buffer(500)`

→ Produces a circular (or inflated) polygon

→ Area $\approx \pi \times 500^2 \approx 785,398 \text{ m}^2$

→ *Notebook Cell [5]: NTU Civil Engineering building — 500m buffer zone*

Warning: `buffer(500)` in EPSG:4326 = 500 “degrees” , NOT meters!

Spatial Relationships: Thinking in Logic

Predicate Logic (The “True/False” of Space):

- Do they touch or overlap? intersects:
- Is A completely inside B? within:
- Does A completely hold B? contains:

Why it matters:

This is the “Filter” for disaster impact assessment.

“Is this bridge inside the fault buffer zone?” → True / False

Spatial logic = the geometric version of an IF statement

Spatial Join (sjoin): The Ultimate Superpower

- 500 Shelter points + River Buffer Polygons Scenario:
- Manual checks in GIS software? Traditional:
- One line of code GeoPandas:

```
gpd.sjoin(shelters, river_buffer, predicate='within')
```

→ Each shelter automatically gets its flood risk status

→ *Notebook Cell [6]: Simulated spatial join walkthrough*
Your shelter data now has a new column: risk_level!

The “Black Box” Buster: `.explore()`

- Use `.explore()` instead. Stop using `.plot()`.
It creates a dynamic, interactive map inside Jupyter.
- `gdf.explore(column= 'risk_score' , cmap= 'OrRd' )` Code:
You can zoom, click, and inspect feature attributes.

Student Mission:

After every cell execution, check the map with `.explore()`
→ *Notebook Cell [7]: Visual Diagnostic Lab — target point + buffer zone*

“Hands on the map. Verify what the buffer actually covers.”

Infrastructure First: Modularity

- “Do the whole analysis.” Don’t ask AI to:
- “Create a function that takes a GDF and returns its 1km buffer.” Ask AI to:

Logic Separation:

Analysis functions (spatial_helper.py) vs Visualization (plot_utils.py)
Each function can be independently tested and verified.

Prompt Strategy:

- ✗ “Analyze all shelters for flood risk”
- ✓ “Write a function: input GDF + distance, output buffered GDF”

Lab 1: The Buffer Unit Trap (30 min)

Goal: Experience the “write one cell, run one cell, inspect one cell” flow

- Load WRA river polygon data (EPSG:3826) → `gdf.head()` to inspect columns Step 1:
- Check CRS → `print(gdf.crs)` → Degrees or meters? Step 2:
- Convert to EPSG:4326 → `buffer(500)` in wrong CRS → observe the result Step 3:
- Compare with correct buffer in EPSG:3826 → `re-buffer(500)` → `.explore()` to verify Step 4:
- Export GeoJSON + screenshot for submission Step 5:

Self-check: Compare area numbers between 3826 vs 4326 buffer — how different are they?

Topology Errors: Figure-8 Polygons

- AI might generate a polygon that “crosses itself” Problem:

Diagnostic: `gdf.is_valid`

→ Returns True / False for each record

Fix: `gdf.make_valid()`

→ Automatically repairs self-intersecting polygons

- Why it matters:

Computers cannot calculate the area of a self-intersecting shape.

Shapely won't throw an error — it just gives the wrong answer!

“No error” does NOT mean “correct”

Windsurf Tip: Jupyter Integration

- write code directly into .ipynb cells Cascade can:
- Strategy:
 1. Let Cascade write ONE cell
 2. Click “Run Cell” to verify immediately
 3. Inspect the output — does it make sense?
 4. Then let Cascade write the next cell

Never let AI write the entire Notebook at once!

“One cell at a time, so you understand every step.”

Data Ingestion via OSM API (osmnx)

- Mapping the world through tags (e.g., bridge=yes) OSM Logic:
- Open, dynamic, and global coverage Why it's better:

Code Example:

```
ox.features_from_address( "Taipei" , tags={ 'bridge' : True},  
dist=1000)
```

- Fetch all bridges or buildings in a specific city Exercise:
→ *Notebook Cell [8]: Fetch NTU-area buildings + geometry repair*

Note: OSM data may mix Point / LineString / Polygon types

The “Captain’s Log” : Markdown Cells

Professional Notebooks = 50% code + 50% explanation

- Use Markdown cells to document:
 - WHY you chose a 5km buffer (not just the code)
 - What you EXPECTED vs. what you GOT
 - Your “diagnostic log” — what the AI did right and wrong

Format Examples:

2. CRS Conversion (Section header)

****Warning****: Degrees and Meters cannot be mixed (Callout)

“Code without explanation is unreadable in 3 months — even to yourself.”

Analyzing Impact: Overlay vs. Join

Join — Attach Attributes

"This bridge is in Zone A"

`gpd.sjoin(bridges, zones)` → Bridge gets `zone_name` column

Overlay — Cut Geometries

"The exact part of the road that is submerged"

`gpd.overlay(roads, floods, how= 'intersection' )` → New geometry

When to use which?

Need attributes → `sjoin`; Need new geometry → `overlay`

`sjoin` ≈ `VLOOKUP` by location; `overlay` ≈ cookie-cutter intersection

Environment Variables in Jupyter

Why use .env files?

API keys (CWA, Gemini) should never be hardcoded

Parameterize: BUFFER_DISTANCES=1000,5000,10000

Jupyter Magic:

```
%load_ext dotenv
```

```
%dotenv
```

→ Automatically loads all variables from .env

➤ **Even during live demos, your keys stay hidden Security:**

“Never hardcode secrets in a Notebook.”

Lab 2: Shelter Flood Risk Analysis (30 min)

Goal: Complete spatial analysis pipeline, entirely in Jupyter

- Load shelters (osmnx or CSV) + river buffer from Lab 1 Step 1:
- CRS alignment — convert both to EPSG:3826 Step 2:
- `gpd.sjoin(shelters, buffer, predicate= 'within' )` Step 3:
- Handle one-to-many — `drop_duplicates() / groupby()` Step 4:
- `.explore()` visualization: red = at-risk, green = safe Step 5:

Self-check: Does `sjoin` produce duplicates? Why?

Case Study: River Flood Shelter Risk

5-Step Workflow:

- Fire Agency Shelters (data.gov.tw CSV) 1. Ingest:
- WRA River Polygons (Shapefile from water.gov) 2. Ingest:
- Rivers → 500m / 1km / 2km (multi-level) 3. Buffer:
- Classify shelters by risk zone (High / Med / Low) 4. Spatial Join:
- Capacity gap analysis by district + export map 5. Summary:
→ *This is the core workflow of the ARIA homework*

Extrapolation: Generalizing the Logic

If it works for Faults & Bridges, it works for:

- Flood risk assessment Rivers × Schools:
- Radius risk Nuclear Plants × Evacuation Zones:
- Access risk after disaster Landslides × Roads:
- (Your Week 2 homework!) Air Pollution × Shelters:

The Pattern is Always the Same:

Ingest → Transform CRS → Buffer → sjoin → Analyze → Export

“Learn one pattern, solve a hundred problems.”

Week 3 Reflection

“Transparency is the key to trust.”

You are no longer just “running AI code” ;
you are orchestrating a spatial logic flow.

Captain’ s Reflection:

- What happens to Buffer results if you skip to_crs? 1.
- What is the biggest difference between sjoin and VLOOKUP?
2.
- Can you now “see through” the GeoPandas code AI writes?
3.

Homework: ARIA (Automated Regional Impact Auditor)

Scenario: Assess flood risk for shelters near rivers

- **Load Fire Agency shelter CSV (data.gov.tw/dataset/73242)**
Task 1:
- **Load WRA river polygons + Multi-level Buffer (500m/1km/2km)** Task 2:
- **sjoin to classify shelter risk (High / Med / Low) + capacity gap** Task 3:
- **Export shelter_risk_audit.json + risk_map.png** Task 4:
- **GitHub Repo + README with AI Diagnostic Log** Task 5:

Submission: Push to GitHub before next class

See Homework-Week3.md for full details